Project Report: Fraud Detection in Banking Transactions

Fundamentals of Database Systems (CSE462a)

Youssef Basem (240919), Moaaz Tamer (245789),
Ramy Emad (241897)
Youssef.bassem1@msa.edu.eg, moaaz.tamer@msa.edu.eg
ramy.emad1@msa.edu.eg
Supervised by: Dr. Ahmed Ayoub

May 7, 2026

Abstract

This document presents a detailed report on the project titled **Fraud Detection in Banking Transactions**. The system is designed to monitor and detect fraudulent activities in banking through transaction tracking, anomaly detection, flagged accounts, and fraud reports. The report covers the problem statement, objectives, database design (ERD and normalization), implementation details including stored procedures, triggers, and transactions, query optimization with indexing and execution plan analysis, data integrity and security measures, and conclusions. The database is implemented in MySQL 8.0 with 11 normalized tables, 20+ records per table, and comprehensive fraud detection logic.

Contents

1 Introduction

1.1 Problem Statement

Financial fraud is a growing concern in the banking industry, costing billions of dollars annually. Traditional manual monitoring is insufficient to handle the volume and velocity of modern banking transactions. Banks need automated systems that can track transactions in real-time, detect anomalous patterns, flag suspicious accounts, and generate investigation reports. This project addresses the need for a robust relational database system that serves as the backbone of a fraud detection platform, enabling analysts to identify, investigate, and resolve fraudulent activities efficiently.

1.2 Objectives

- Design a normalized relational database schema (3NF) for fraud detection in banking.
- Implement transaction tracking with comprehensive metadata (channel, IP, device, location).
- Create configurable fraud detection rules for various anomaly patterns.
- Build stored procedures for automated fraud scanning (high-value, velocity, geographic anomalies).
- Implement triggers for real-time audit logging and alert generation.
- Optimize query performance through strategic indexing for large datasets.
- Enforce data integrity through constraints and role-based access control.
- Generate fraud investigation reports and risk scoring for customers.

1.3 Scope

The system covers the following functional areas:

- **Core Banking**: Customer profiles, bank accounts, branch management.
- **Transaction Monitoring**: Full transaction lifecycle with metadata capture.
- **Fraud Detection**: Configurable rules, automated alert generation, severity classification.
- **Investigation**: Fraud reports, account flagging, risk scoring.
- **Security**: User roles, audit logging, login attempt tracking.

Limitations: The system focuses on the database layer; front-end applications and real-time streaming are out of scope. Machine learning-based detection is represented through configurable rule thresholds rather than actual ML models.

2 Database Design

2.1 Entity-Relationship Diagram (ERD)

The ERD below illustrates the complete database design using Chen's notation (rectangles for entities, diamonds for relationships, ovals for attributes, with cardinality labels). The system consists of 11 entities organized into three logical layers: Core Banking (Branches, Customers, Accounts), Transaction Processing (Transactions), and Fraud Detection (FraudAlerts, FraudRules, FlaggedAccounts, FraudReports, Users, AuditLog, LoginAttempts).

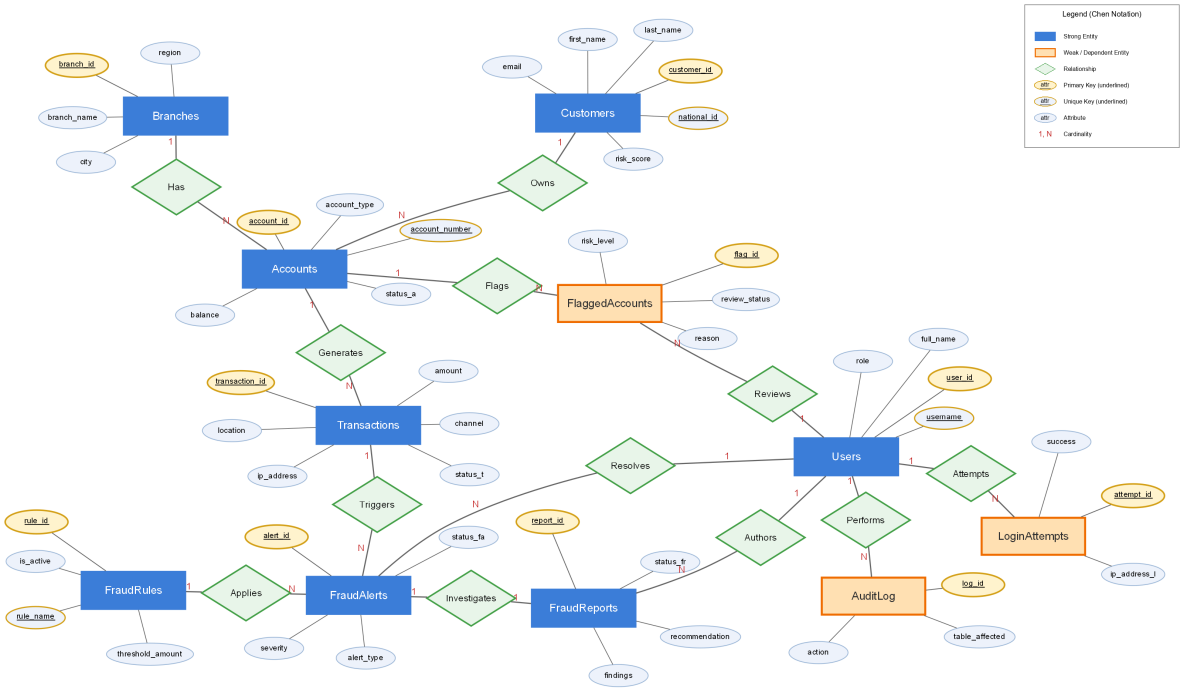

Figure 1: Entity-Relationship Diagram — Chen's Notation

Figure 2 presents the same schema using Crow's Foot notation, which provides a more detailed physical view including data types, nullability constraints, and foreign key references for each table.

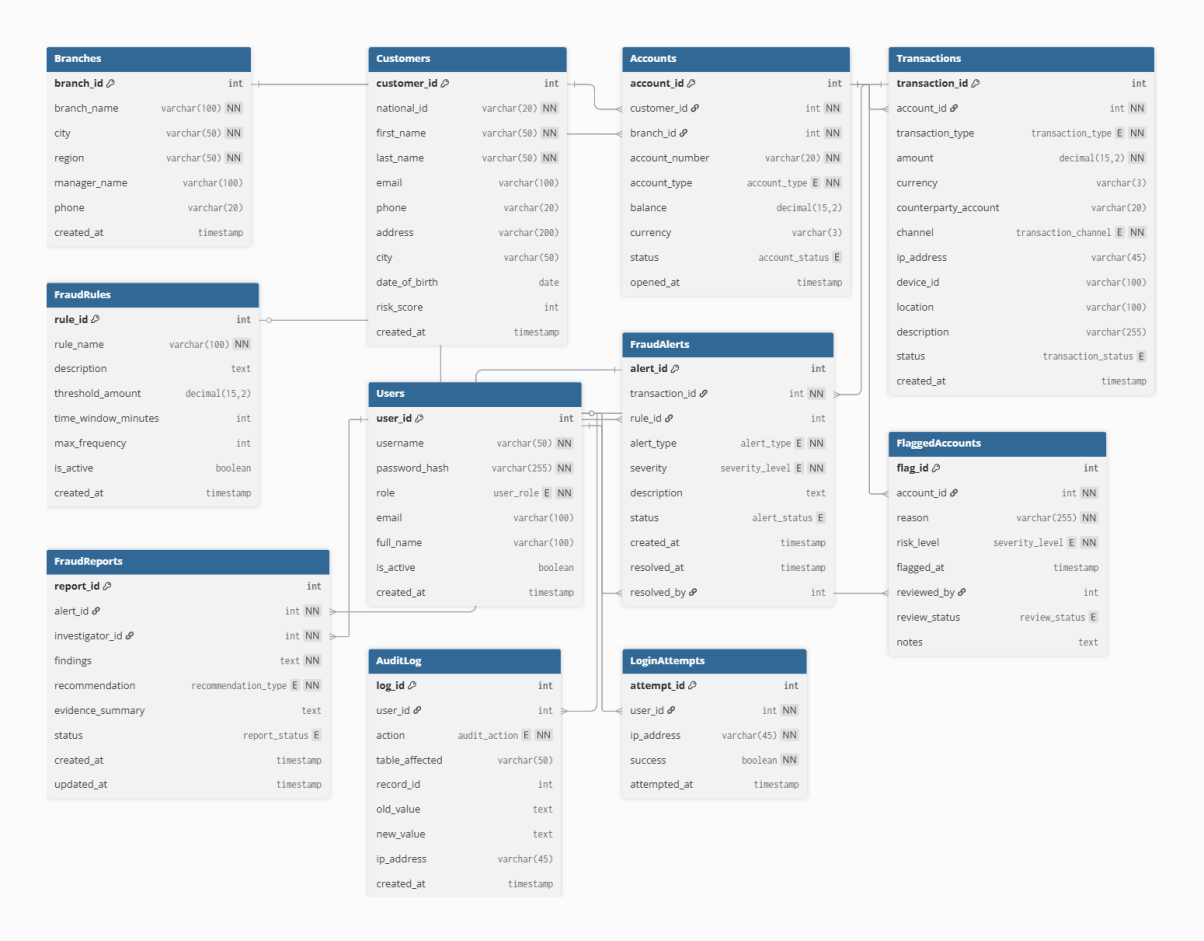

Figure 2: Entity-Relationship Diagram — Crow's Foot Notation

Key Relationships:

- Branches (1) → (N) Accounts: Each branch hosts multiple accounts.
- Customers (1) → (N) Accounts: A customer can have multiple accounts.
- Accounts (1) → (N) Transactions: Each account has many transactions.
- Transactions (1) → (N) FraudAlerts: A transaction can trigger multiple alerts.
- FraudRules (1) → (N) FraudAlerts: Rules generate alerts.
- FraudAlerts (1) → (1) FraudReports: Each alert may have one investigation report.
- Users resolve alerts, review flagged accounts, and author reports.

2.2 Normalization

The database is designed in **Third Normal Form (3NF)**:

1NF: All tables have atomic values. No repeating groups. Each column contains a single value (e.g., customer name is split into first_name and last_name).

2NF: All non-key attributes are fully functionally dependent on the entire primary key. Since all tables use single-column surrogate primary keys (AUTO_INCREMENT), partial dependencies are eliminated by design.

3NF: No transitive dependencies exist.

- Branch information is separated from Accounts (branch_id FK instead of embedding branch details).
- Customer data is separated from Accounts.
- Fraud rules are in a separate table referenced by FraudAlerts via rule_id FK.
- User information (investigators, reviewers) is referenced via FK rather than stored inline.

No denormalization was applied as the schema is efficient for the expected query patterns. Views are used to provide denormalized read access without compromising the normalized base tables.

2.3 Schema Design

The complete relational schema (detailed in Figure 2) consists of 11 tables with the following key design decisions:

- **ENUM types** for status fields (account status, alert severity, transaction type) to enforce domain constraints at the database level.
- **Surrogate keys** (AUTO_INCREMENT) for all primary keys for consistent referencing.
- **Composite indexes** on frequently queried column combinations.
- **CHECK constraints** on risk_score (0–100) and amount (> 0).
- **SHA2 hashing** for password storage in the Users table.

3 Implementation

3.1 Database Tables

The system comprises 11 tables:

Table	Key Columns	Purpose
Branches	branch_id (PK), branch_name, city, region	Bank branch locations
Customers	customer_id (PK), national_id (UK), risk_score	Customer profiles with fraud risk scores
Accounts	account_id (PK), customer_id (FK), branch_id (FK), status	Bank accounts linked to customers and branches
Transactions	transaction_id (PK), account_id (FK), amount, channel, ip_address, location	Complete transaction records with metadata
FraudRules	rule_id (PK), rule_name (UK), threshold_amount, time_window	Configurable fraud detection rules
FraudAlerts	alert_id (PK), transaction_id (FK), rule_id (FK), severity, status	Alerts generated by fraud detection
FlaggedAccounts	flag_id (PK), account_id (FK), risk_level, review_status	Accounts under investigation
FraudReports	report_id (PK), alert_id (FK), investigator_id (FK), recommendation	Investigation reports
Users	user_id (PK), username (UK), password_hash, role	System users (Admin, Analyst, Teller, Auditor)
AuditLog	log_id (PK), user_id (FK), action, table_affected	System audit trail
LoginAttempts	attempt_id (PK), user_id (FK), ip_address, success	Login tracking for security

Each table contains at least 20 sample records with realistic banking data, including intentionally fraudulent patterns such as:

- Rapid successive transfers (3 transfers within 10 minutes)

- Large international wires during midnight hours (1–5 AM)
- Geographic impossibilities (Dubai to Nairobi in 45 minutes)
- Potential structuring (amounts near $10,000 reporting threshold)

3.2 SQL Queries

CRUD Operations:

Listing 1: SELECT - Active accounts with customer info

```sql
SELECT c.first_name, c.last_name, a.account_number,
       a.account_type, a.balance, a.status
FROM Customers c
JOIN Accounts a ON c.customer_id = a.customer_id
WHERE a.status = 'Active' ORDER BY a.balance DESC;
```

Listing 2: UPDATE - Resolve a fraud alert

```sql
UPDATE FraudAlerts
SET status = 'Resolved', resolved_at = NOW(), resolved_by = 3
WHERE alert_id = 3;
```

JOIN Operations:

Listing 3: Multi-table JOIN - Full fraud investigation view

```sql
SELECT c.first_name, c.last_name, c.risk_score,
       a.account_number, t.amount, t.channel, t.location,
       fa.alert_type, fa.severity, fa.status AS alert_status
FROM Customers c
INNER JOIN Accounts a ON c.customer_id = a.customer_id
INNER JOIN Transactions t ON a.account_id = t.account_id
INNER JOIN FraudAlerts fa ON t.transaction_id = fa.transaction_id
ORDER BY fa.severity DESC, t.amount DESC;
```

Aggregate Functions:

Listing 4: Fraud amounts by alert type with aggregates

```sql
SELECT fa.alert_type, COUNT(*) AS alert_count,
       SUM(t.amount) AS total_flagged_amount,
       AVG(t.amount) AS avg_flagged_amount
FROM FraudAlerts fa
JOIN Transactions t ON fa.transaction_id = t.transaction_id
GROUP BY fa.alert_type ORDER BY total_flagged_amount DESC;
```

Subqueries:

Listing 5: Accounts with transactions exceeding 3x their average

```sql
SELECT a.account_number, t.amount, t.created_at, t.channel
FROM Transactions t
JOIN Accounts a ON t.account_id = a.account_id
WHERE t.amount > 3 * (
    SELECT AVG(t2.amount) FROM Transactions t2
    WHERE t2.account_id = t.account_id
) ORDER BY t.amount DESC;
```

3.3 Stored Procedures, Triggers, Transactions

Stored Procedures (5 implemented):

1. `sp_detect_high_value_fraud(threshold)` – Scans all transactions above a threshold amount, automatically creates fraud alerts with appropriate severity levels (Critical $\geq$ \$50k, High $\geq$ \$20k, Medium $\geq$ \$10k). Uses a cursor to iterate through unprocessed transactions.

2. `sp_detect_velocity_fraud(account_id, time_window, max_count)` – Detects rapid-fire transactions by counting transactions within a specified time window. Generates alerts when the count exceeds the maximum allowed frequency.

3. `sp_generate_fraud_report(alert_id, investigator_id, findings, recommendation)` – Creates a formal investigation report linked to a fraud alert, validates the alert exists, and updates the alert status to Under_Review.

4. `sp_update_risk_score(customer_id)` – Calculates a dynamic risk score (0–100) based on confirmed frauds ($\times$25), open alerts ($\times$10), and total flagged amount (scaled by \$10k increments).

5. `sp_fund_transfer(from, to, amount)` – Performs a safe fund transfer within a transaction, checking account status and balance before proceeding. Automatically flags transfers $\geq$ \$10,000.

Listing 6: Fund Transfer Stored Procedure (excerpt)

```
CREATE PROCEDURE sp_fund_transfer(
    IN p_from INT, IN p_to INT, IN p_amount DECIMAL(15,2))
BEGIN
    DECLARE v_balance DECIMAL(15,2);
    DECLARE v_from_status VARCHAR(10);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        BEGIN ROLLBACK; RESIGNAL; END;
    START TRANSACTION;
    SELECT balance, status INTO v_balance, v_from_status
        FROM Accounts WHERE account_id = p_from FOR UPDATE;
    IF v_from_status != 'Active' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Source account is not active';
    END IF;
    IF v_balance < p_amount THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Insufficient balance';
    END IF;
    UPDATE Accounts SET balance = balance - p_amount
        WHERE account_id = p_from;
    UPDATE Accounts SET balance = balance + p_amount
        WHERE account_id = p_to;
    COMMIT;
END
```

Triggers (3 implemented):

1. `trg_after_transaction_insert` – Fires after each new transaction. Logs high-value transactions ($\geq$ \$10k) and midnight transactions ($\geq$ \$5k between 1–4 AM) to the AuditLog.

2. `trg_after_alert_update` – Fires when a fraud alert status changes to a terminal state (Confirmed_Fraud, Resolved, False_Positive). Records the status change in AuditLog.

3. `trg_account_status_change` – Fires when an account status changes (e.g., Active → Frozen). Logs the change for audit compliance.

ACID Transactions:

1. Fund transfer between accounts with balance verification and rollback on failure.

2. Batch fraud alert status update with transactional consistency.

3. Account freeze with cascading flagged account creation.

4 Query Optimization

Strategic indexing was implemented to optimize the most critical query patterns:

Index	Target Query Pattern	EXPLAIN Result
`idx_transactions_account_date` (account_id, created_at)	Transactions by account within date range	type=range, Using index condition
`idx_transactions_amount` (amount)	High-value transaction scans	type=range, Using index condition
`idx_fraud_alerts_status_severity` (status, severity)	Alert dashboard filtering	type=ref, key=idx (exact match)
`idx_customers_national_id` (national_id)	Customer lookup by national ID	type=const (single row)

EXPLAIN Analysis Results:

- **Account transaction lookup**: Uses composite index `idx_transactions_account_date` with `type=range`, scanning only 1 row instead of full table scan.
- **High-value scan**: Uses `idx_transactions_amount` with `type=range`, filtering to 10 candidate rows.
- **Critical open alerts**: Uses `idx_fraud_alerts_status_severity` with exact ref lookup, returning 1 matching row directly.
- **National ID lookup**: Uses the UNIQUE constraint index with `type=const`, the fastest possible access method.

Additional optimization techniques:

- **Views** (vw_fraud_dashboard, vw_suspicious_transactions, vw_account_risk_profile) provide pre-computed denormalized read access without compromising base table normalization.
- **Covering indexes** on high-frequency query columns reduce disk I/O.
- **FOR UPDATE** locking in the fund transfer procedure prevents phantom reads during concurrent access.

5 Data Integrity and Security

5.1 Constraints and Data Validation

- **Primary Keys**: All 11 tables use AUTO_INCREMENT surrogate keys.
- **Foreign Keys**: 12 foreign key constraints enforce referential integrity across tables. ON DELETE CASCADE is used for Customers → Accounts.
- **UNIQUE Constraints**: national_id, email (Customers), account_number (Accounts), username (Users), rule_name (FraudRules).
- **CHECK Constraints**: risk_score BETWEEN 0 AND 100, amount > 0.
- **ENUM Types**: Enforce valid values for status fields, transaction types, channels, account types, severity levels, and user roles.
- **NOT NULL**: Applied to all critical fields (names, amounts, types).
- **DEFAULT Values**: Timestamps default to CURRENT_TIMESTAMP, risk_score defaults to 0, account status defaults to 'Active'.

5.2 Access Control and Security Measures

Role-Based Access Control (RBAC):

Role	Permissions
fraud_admin	Full access to all tables and operations
fraud_analyst	SELECT on all tables; INSERT/UPDATE on FraudAlerts, FraudReports, FlaggedAccounts, AuditLog
bank_teller	SELECT only on Transactions, Customers, Accounts

Security Measures:

- **Password Hashing**: User passwords stored using SHA2-256 hashing.
- **Login Monitoring**: LoginAttempts table tracks all login attempts with IP addresses. Multiple failed attempts (e.g., 5 failures in 5 minutes) trigger security alerts.
- **Audit Trail**: AuditLog captures all data modifications with before/after values, user ID, and timestamp for forensic analysis.
- **Parameterized Procedures**: Stored procedures use parameterized inputs, preventing SQL injection attacks.

6 Results and Discussion

6.1 Testing and Validation

The system was validated through the following tests:

- **Data Integrity**: All 11 tables successfully created with 20+ records each. Foreign key constraints verified through cascading operations.
- **Query Correctness**: All SELECT, JOIN, aggregate, and subquery operations return expected results. The fraud dashboard view correctly reports 3 open alerts, 9 under review, 5 confirmed frauds, and $216,000 in confirmed fraud amount.
- **Stored Procedures**: All 5 procedures tested successfully—fund transfers correctly debit/credit accounts, fraud detection scans identify suspicious transactions, risk scoring produces meaningful scores.
- **Triggers**: All 3 triggers verified—AuditLog entries automatically created on transaction inserts and status changes.
- **Index Performance**: EXPLAIN analysis confirms all strategic indexes are utilized by the query optimizer, eliminating full table scans.
- **ACID Compliance**: Fund transfer transactions correctly rollback on insufficient balance, maintaining data consistency.

6.2 Challenges and Limitations

- **Real-time Processing**: The current design uses batch-oriented stored procedures rather than real-time event streaming, which would be needed for production deployment.
- **Rule Complexity**: The fraud rules table supports threshold-based detection but lacks support for complex pattern matching (e.g., graph-based fraud networks).
- **Scalability**: For production datasets with millions of transactions, additional partitioning strategies and materialized views would be necessary.
- **Machine Learning Integration**: The risk scoring is formula-based; integrating ML models would require an external scoring service and API layer.

7 Conclusion

This project successfully demonstrates a comprehensive fraud detection database system for banking transactions. The system implements 11 normalized tables in 3NF, 5 stored procedures for automated fraud detection and investigation workflows, 3 triggers for audit compliance, strategic indexing that eliminates full table scans, and role-based access control for security.

Key achievements include:

- A configurable rule-based fraud detection engine
- Dynamic customer risk scoring based on fraud history
- Complete audit trail for regulatory compliance
- Query optimization with verified EXPLAIN execution plans

Future improvements could include:

- Integration with machine learning models for predictive fraud detection
- Real-time transaction streaming using MySQL event scheduling
- Geographic visualization of fraud patterns
- API layer for integration with banking front-end applications

8 References

1. R. Elmasri and S. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
2. A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.
3. MySQL 8.0 Reference Manual, Oracle Corporation, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/
4. S. Panigrahi, A. Kundu, S. Sural, and A. K. Majumdar, "Credit card fraud detection: A fusion approach using Dempster-Shafer theory and Bayesian learning," *Information Fusion*, vol. 10, no. 4, pp. 354–363, 2009.
5. R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002.